

PUC CAMPINAS

Pontifícia Universidade Católica de Campinas
Escola Politécnica

JavaScript, DOM e Validações de Formulários

Introdução as Tecnologias para Web

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

PONTO DE PARTIDA

Um botão sozinho não sabe o que fazer

```
<button class="btn btn-primary">Salvar</button>
```

O que acontece quando o usuário clica?

Sem uma ação programada: nada.

VISÃO GERAL

Os dados percorrem um caminho



QUESTÃO-GUIA

Como transformar dados digitados em uma estrutura válida e pronta para ser enviada?

Cada tecnologia tem uma responsabilidade

HTML

Estrutura

campos, botões, títulos

CSS / Bootstrap

Aparência

layout, cores, responsividade

JavaScript

Comportamento

eventos, lógica, alterações

JavaScript executa em ambientes diferentes

AGORA

Navegador
DOM • eventos • interface

DEPOIS

Node.js
servidor • rotas • backend

Nesta aula, o foco é exclusivamente o navegador.

CONFIGURAÇÃO

Separe estrutura e comportamento

```
<script src="script.js" defer></script>
```

src indica o arquivo JavaScript
defer aguarda o HTML ser interpretado
O script pode acessar os elementos quando executar

```
projeto/
├── index.html
├── style.css
└── script.js
```

Exemplo 1 - Observe o programa pelo console

```
console.log("JavaScript carregado!");

console.log(nome, idade);
```

- Abra F12 ou Inspecionar
- Selecione a aba Console
- Recarregue a página
- Leia mensagens e erros

HÁBITO

Erros no console são pistas sobre o estado do programa.

Use const por padrão e let para mudanças

```
const universidade = "PUC-Campinas";
let quantidade = 2;

quantidade = 3;
```

```
const nome = "João";    //
string
const idade = 20;       //
number
const ativo = true;     //
boolean
```

Não usaremos var neste módulo.

Condições e funções organizam o comportamento

```
function verificarPrioridade(prioridade) {
  if (prioridade === "alta") {
    return "Atenção imediata";
  }

  return "Fluxo normal";
}
```

- === compara valor e tipo
- return encerra a função
- parâmetros recebem dados
- o nome expressa a intenção

O DOM torna o HTML manipulável



O HTML inicial não precisa permanecer fixo.

Primeiro selecione; depois altere

```
const titulo = document.getElementById("titulo");

const campo = document.querySelector("#nome");
const botao = document.querySelector(".btn-salvar");
```

LEITURA

seleciona um id • . seleciona uma classe

Selecionar permite alterar conteúdo e aparência

```
titulo.innerText = "Sistema de Demandas";

mensagem.innerText = "Dados salvos!";
mensagem.classList.remove("d-none");
mensagem.classList.add("text-success");
```

innerText altera o texto
classList altera classes CSS
Bootstrap continua cuidando da aparência

Eventos ligam ações a funções



Eventos frequentes: click • submit • change • input

addEventListener define a resposta ao evento

```
const botao = document.querySelector("#btnSalvar");

botao.addEventListener("click", function () {
  alert("Botão clicado!");
});
```

LEIA EM VOZ ALTA

No clique do botão, execute esta função.

value lê o conteúdo do campo

```
<input type="text" id="titulo">
```

```
const campoTitulo =

document.querySelector("#titulo");

const titulo = campoTitulo.value;
```

Elemento HTML ≠ valor armazenado no elemento

SÍNTESE DA PRIMEIRA PARTE

A interface reage; os dados exigem proteção

JÁ SABEMOS

selecionar • ouvir eventos • ler valores • alterar o DOM

NOVA PERGUNTA

Podemos aceitar e enviar qualquer valor digitado pelo usuário?

O evento submit pertence ao formulário

```
<form id="formDemanda">
  <input id="titulo">
  <button
type="submit">Salvar</button>
</form>
```

```
const formulario =

document.querySelector("#formDemanda")
;

formulario.addEventListener("submit",
  function (event) {
    console.log("Enviado");
  }
);
```

preventDefault bloqueia o envio padrão

```
formulario.addEventListener("submit", function (event) {
  event.preventDefault();

  // A partir daqui, nós controlamos o fluxo.
});
```

Sem preventDefault: o navegador realiza o envio tradicional e a página pode recarregar.

trim remove espaços antes da validação

```
const titulo = campoTitulo.value.trim();
const descricao = campoDescricao.value.trim();
const prioridade = campoPrioridade.value;
```

Entrada original: " "
Após trim(): ""

VALIDAÇÃO

Valide antes de continuar

```
if (titulo === "") {
  erroTitulo.innerText = "O título é obrigatório.";
  return;
}

if (descricao.length < 10) {
  erroDescricao.innerText =
    "Informe ao menos 10 caracteres.";
  return;
}
```

PAPEL DO RETURN

Interromper a função quando a regra não for satisfeita.

VALIDAÇÃO

A validação ocorre em camadas complementares

HTML

Regras declarativas

required • type=email • minLength

JavaScript

Regras da interface

combinações • mensagens • experiência

Backend

Regra obrigatória

segurança • consistência • confiança

REGRA DE OURO

Nunca confie apenas na validação feita no navegador.

Exiba mensagens próximas ao campo

```
<input id="titulo" class="form-
control">
<div id="erroTitulo"
     class="text-danger small mt-
1"></div>
```

```
erroTitulo.innerText =
  "O título é obrigatório.";

campoTitulo.classList.add("is-
invalid");
```

- Evite depender somente de alert()
- Indique exatamente onde está o erro
- Limpe mensagens antigas a cada tentativa

Mostre todos os erros de uma vez

```
let formularioValido = true;

if (titulo === "") {
  erroTitulo.innerText = "Campo obrigatório.";
  formularioValido = false;
}

if (descricao.length < 10) {
  erroDescricao.innerText = "Texto muito curto.";
  formularioValido = false;
}

if (!formularioValido) return;
```

Objetos agrupam dados relacionados

```
const demanda = {
  titulo,
  descricao,
  prioridade,
  tipo
};
```

LEITURA CONCEITUAL

Esses quatro valores, juntos, representam uma Demanda.

A interface coleta campos; o domínio organiza significado.

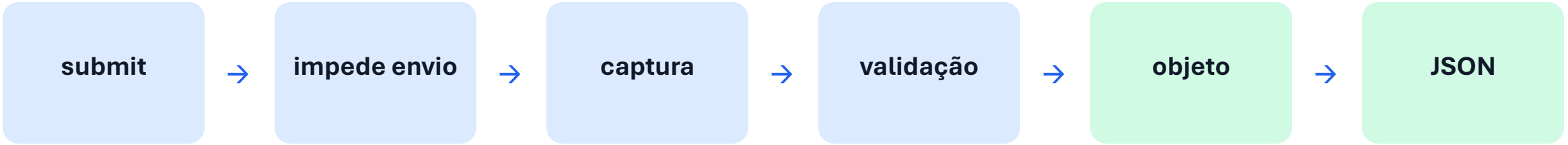
JSON representa dados para comunicação

```
const json = JSON.stringify(demanda, null, 2);
console.log(json);
```

```
{
  "titulo": "Corrigir login",
  "descricao": "Falha ao acessar o sistema",
  "prioridade": "alta",
  "tipo": "bug"
}
```

Objeto JavaScript → JSON → futura requisição HTTP → backend

Dados válidos viram objeto e JSON



AINDA NÃO

Nenhum dado saiu do navegador. Preparamos o conteúdo que será enviado depois.

SÍNTESE E CONTINUIDADE

Dados prontos seguem para o backend

Usuário → Formulário → Validação → Objeto / JSON → **BACKEND**

PRÓXIMA PERGUNTA

Como o backend recebe, valida novamente e processa esses dados?